QoRNet: Timing Prediction for VLSI Circuits (Track II, Sub-Track I)

Cory Brynds DeAndre Hendrix Youssef Samwel Pablo Rodriguez

Abstract

QoRNet is a deep learning-based framework for predicting post-implementation circuit delay from a pre-synthesis RTL representation. We extract vector-based features to train baseline regression models and graph-based features to train a Graph Neural Network (GNN). Using an open-source RTL-to-implementation flow to generate ground truth timing reports, we compare the vector- and graph-based models and quantify accuracy and speedups relative to running a full synthesis and physical implementation flow. The repository containing QoRNet's data collection, training, and inference pipelines can be found at https://github.com/cbrynds/PPA-Estimation-Project.

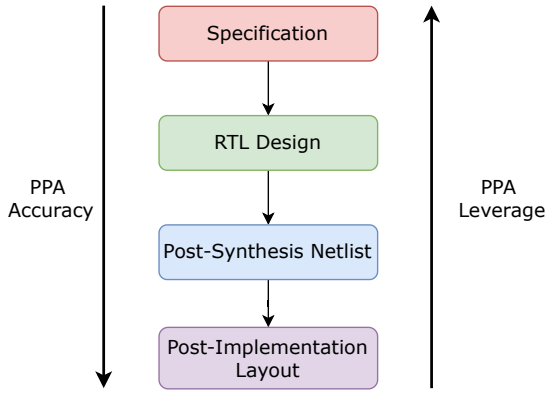


Figure 1. VLSI design flow

1. Introduction

The design of the digital circuits making up microprocessors and other logic devices has seen a tremendous increase in productivity through automation and tooling. Early digital designers drew out individual transistors by hand, and now modern Hardware Description Languages (HDLs) allow designers to create chips with millions or even billions of transistors. Hardware design involves many different trade-offs between power, performance, and area (PPA), and small alterations or even differences in coding style can have a dramatic impact on a chip's final PPA. Furthermore, a designer has limited visibility into the impact of their design choices prior to synthesis and implementation, the processes by which an RTL design is converted to a gate-level representation and then mapped to a chip layout, respectively. At each stage in this process, the design undergoes iterative optimizations to meet user-specified PPA constraints. Synthesis and implementation are prohibitively expensive for large-scale RTL designs, and the resulting PPA of an optimized netlist is hard to predict from the original RTL code.

Designers have the greatest ability to affect PPA early in the design flow, but they have the least visibility into what the final PPA will be (See Figure 1). Even between logic synthesis and physical implementation, there can be up to a **60% discrepancy** (Gandham et al., 2025) between measured timing. To provide more visibility to the designer at the RTL stage of development, we propose a deep learning framework to estimate post-implementation timing.

2. Related Work

Here, we survey prior literature involving the use of neural networks for timing prediction in VLSI circuits.

2.1. OpenABC-D: A Large-Scale Dataset For Machine Learning Guided Integrated Circuit Synthesis

OpenABC-D (Chowdhury et al., 2021) is a large-scale, labeled dataset for logic synthesis and IC design tailored to AI/ML applications, including developing, evaluating, and benchmark ML-guided synthesis. The dataset was produced from 1,500 synthesis runs and consists of 870,000 And-Inverter-Graphs (AIG) in the form of intermediate and final synthesis outputs. The authors introduced benchmark methodologies and evaluated existing state-of-the-art GCN models against the dataset to predict the Quality-of-Results (QoR) of synthesis recipes for different IP cores. It was found that GCN models can learn functional and structural information from the AIGs. OpenABC-D established the foundation for ML pipelines in the EDA community.

2.2. SNS's Not a Synthesizer: A Deep-Learning-Based Synthesis Predictor

SNS (Xu et al., 2022) is a deep learning-based synthesis predictor that approximates the power, performance, and area (PPA) as well as the timing characteristics of a design. The approach is based on transformer networks and multi-layer perceptrons to perform inference on the global properties of a design. The model can predict a design's PPA characteristics in a fraction of the time it takes to run a full synthesis flow with an average RRSE of 0.4998. The path-based representation utilized in SNS is scalable to large designs, however it often fails to capture global node interactions leading to high error rates in timing predictions.

2.3. How Good is Your Verilog RTL Code? A Quick Answer from Machine Learning

This work estimates the TNS and dynamic power consumption of a circuit based on its RTL representation (Sengupta et al., 2022). QAML uses Verilator to parse the source RTL. The AST is flattened and saved to an XML format, after which QAML prepares the input dataset for the model using two methods: vector-based feature extraction and graph-based feature extraction. Then, both vector-based and graph-based models are evaluated to see which model is able to learn the structural transformations applied by logic synthesis. It reports a high accuracy in predicting dynamic power consumption and TNS, with a speedup of six orders of magnitude over a full commercial synthesis and placement flow.

2.4. Limitations of Prior Works

We identified two technical gaps in related works that limit the ability of models to make accurate predictions and we propose potential solutions to these problems:

1. **Early Prediction**: OpenABC (Chowdhury et al., 2021) and SNS (Xu et al., 2022) are trained only with data from the early stages of IC design pipelines; in particular, they lack cell and routing placement information. This limits their usefulness to accurately model physical timing characteristics of synthesized designs (Gandham et al., 2025). We incorporate post-placement-and-routing timing information in the training set to better capture timing characteristics of later IC design stages.

2. **Structural Compression**: SNS utilized a path-based model that captures local timing paths but is prone to missing global node interactions. This leads the model to poorly predict WNS and TNS timing results in large designs. To better capture global interactions, we focus on using attention-based graph regression networks instead.

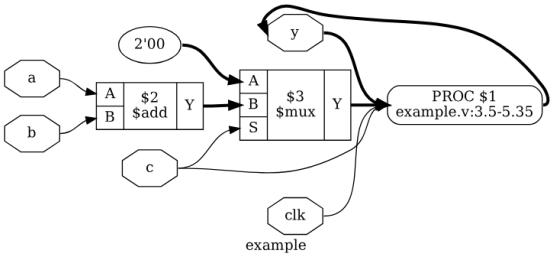


Figure 2. Example of a Yosys-generated AST.

3. Background

3.1. Timing Analysis in Digital Circuits

When implementing digital circuits, static timing analysis (STA) is used to check whether electrical signals can propagate through each timing path. For each path, the STA tool will compare the **required time** (the time in which the signal must be valid) against the **arrival time** (the time in which the signal actually arrived). The difference between required time and arrival time is the slack:

$$T_{slack} = T_{required} - T_{arrival}$$

If the slack is positive, the path meets timing. If it is negative, this path violates timing. Two metrics used in STA are

1. **Worst Negative Slack (WNS)**: the minimal slack across all timing paths in the design. If negative, it is the worst timing violation. If positive, it is the minimum margin the design meets timing by.

2. **Total Negative Slack (TNS)**: the sum of all of the negative slacks across violating paths. This states how widespread timing violations are in a design.

3.2. Intermediate Representations for Hardware Design

During the logic optimization process, an RTL design will undergo many structural transformations before reaching the final netlist. These structural transformations happen at multiple different intermediate representations, or IRs. One such IR used by Yosys (Wolf et al., 2013) and other tools is the Abstract Syntax Tree, which is a tree-like representation of RTL code where each node represents a language primitive and edges model the flow of data. Figure 2 shows an AST representing the Verilog expression

$$y = c?(a + b) : (2'b00)$$

Nodes include inputs a,b,c, and clk; constant values such as 2'b00; arithmetic operations $add and $mux; and output y.

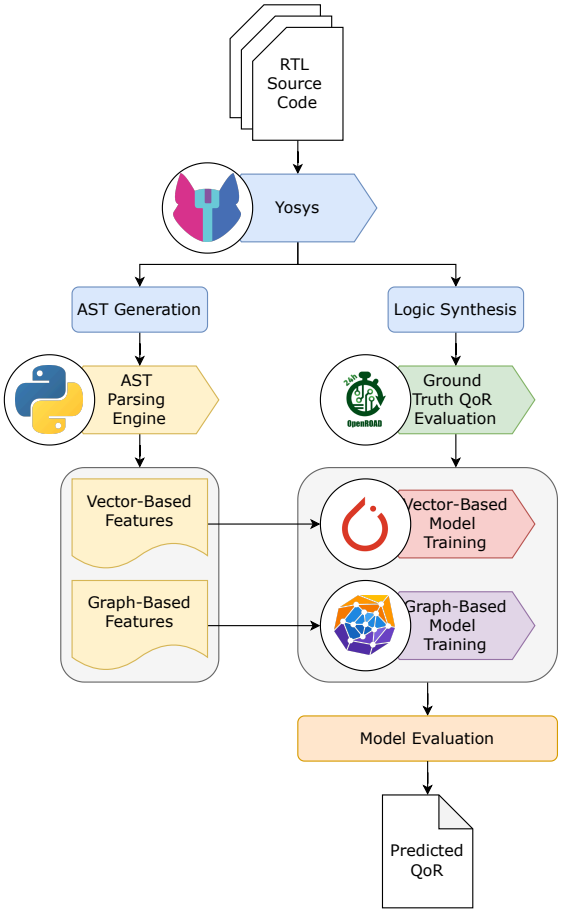


Figure 3. QoRNet implementation flow

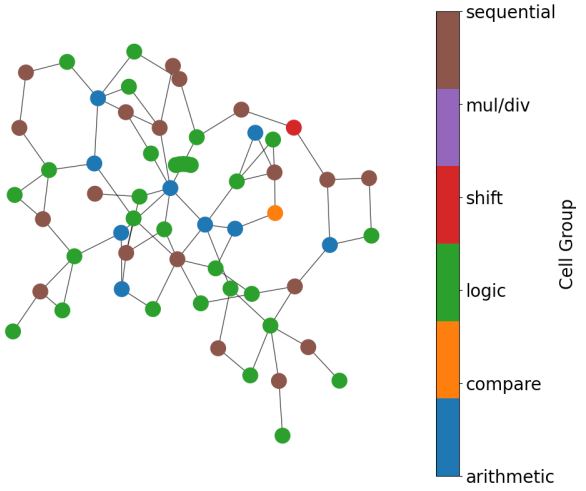


Figure 4. Example of an extracted graph for ss_pcm from the OpenCores (ope) benchmark.

4. Implementation

Figure 3 outlines our implementation flow. First, a dataset of RTL code is used as input to Yosys (Wolf et al., 2013). On the left path of the flow, Yosys generates the AST for each design and passes it to an AST parsing engine, our own implementation. This parsing engine produces two outputs: a tensor of vector-based features and graph-based features similar to (Sengupta et al., 2022). In the right path of the flow, Yosys is used to perform logic synthesis on the RTL code, transforming it into an optimized gate-level netlist. This netlist is passed to OpenROAD, a placement-and-routing (PnR) tool that will produce the ground truth metrics for delay that our model attempts to learn.

In the model training block, two classes of models are trained. The vector-based models include a linear regression and an MLP model. The graph-based model is a Graph Attention Network implemented in PyTorch Geometric (Fey & Lenssen, 2019). After all models are trained and tested, a separate process compares the training and testing results to evaluate which architecture performed the best on the timing prediction task. The QoR results of the most accurate model are returned. For the work, we are focusing on predicting WNS and TNS.

4.1. AST Parsing

For parsing the AST generated by Yosys, we follow the approach described in (Sengupta et al., 2022). Features are split up into vector-based and graph-based features, which are used to train two separate classes of models.

4.1.1. Vector-Based Feature Extraction

Circuit features are extracted to be used with a non-graph-based ML model (e.g. a model processing a flattened vector like an MLP). The extraction engine identifies every register assignment and back-trace the cone of logic responsible for driving the register. Once all the cones of logic are identified, a procedure will compute per-tree a vector of 12 features, summarized in Table 1. A two-dimensional feature vector is constructed from a distribution of all 12 features across all the logic cones in the design. Additional metrics were added to help the model learn the impact of diverse cell types within each HDL design.

4.1.2. Graph-Based Feature Extraction

In graph-based feature extraction, register assignments are again back-traced to acquire the driving logic cones. All registers then become nodes in the graph, in addition to arithmetic operators, boolean logic, constant registers, I/O pins, and muxes. Meanwhile, edges are the buses between node elements (e.g. an arithmetic node will have edges representing its operands). Table 1 summarizes the node

Table 1. Summary of design features extracted from Yosys Abstract Syntax Tree

Feature Set	Feature	Type
Vector-based	Total input bits	Global
	Total output bits	Global
	Total register bits	Global
	Total logic operator bits	Global
	Total adder/subtractor bits	Global
	Total multiplier bits	Global
	Total comparator bits	Global
	Total boolean logic bits	Global
	Total Memory R/W Addr. Bits	Global
	Memory R/W Addr. Width	Global
	Average tree depth	Global
	Average wire width	Global
Graph-based	Total input bits	Node
	Total output bits	Node
	Encoded logic type	Node
	Encoded node type	Node
	# of neighboring nodes	Node
	Edge bitwidth	Edge
	Source-dest. pair type	Edge
	Edge type	Edge

and edge annotations for the extracted graph-based features.

4.2. Model Architectures

We evaluate three different deep learning models to find the best model architecture for learning post-placement-and-routing WNS/TNS from an RTL representation.

4.2.1. Vector-Based Models

Vector-based modeling begins with our dataset construction script merge_vector_truth.py, which merges the pre-synthesis RTL features in Table 1 with the synthesis and implementation recipe features and their corresponding ground-truth WNS/TNS labels. Each sample represents one RTL design under one synthesis/implementation recipe, paired with the resulting post-implementation timing metrics.

Separate single-output models were trained independently to predict WNS and TNS targets as these metrics differ largely in numeric scale, which prevented larger TNS values from dominating the loss.

As a simple baseline, we implemented linear regression, opting for a single PyTorch linear layer rather than using a closed-form least squares solution to keep the training, evaluation, and artifact generation flows aligned between models. Because the dataset consisted of many of the same designs synthesized using slightly different synthesis recipes, we utilize a GroupKFold by design name so that all recipes for a held-out design remain in the same fold. This prevents leakage between training and testing and provides a more realistic estimate of how the model generalizes to unseen designs. Within each fold, numeric inputs are standardized, categorical recipe features are one-hot encoded, and two independent linear models are trained for WNS and TNS. The linear model hyperparameters are summarized in Table 3, and the model is expressed as

$$\hat{Y} = Xw + b$$

The MLP models keeps the same data split methodology but replaces the single linear layer with a 2-layer fully connected network. The model consists of two hidden layers with 64 and 32 hidden nodes, ReLU activation, and dropout of 0.1. Its training hyperparameters are summarized in Table 3. Standardization here had a very noticeable improvement to the model stability, especially for TNS. The MLP model can be expressed as below.

$$\hat{y} = W_3\sigma\left(W_2\sigma\left(W_1x + b_1\right) + b_2\right) + b_3$$

Both vector-based models are optimized using mean-squared error loss:

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

4.2.2. Graph Attention Network

To learn the interaction between different nodes in the graph representation of each RTL circuit, we implement a Graph Attention Network (GAT) (Brody et al., 2022). A GAT implements an attention mechanism and weights the importance of neighbors when learning the node feature embedding. The message-passing GAT is configured as a graph-level regression task, shown in Figure 5. Each graph input's nodes/edges have a set of numeric features (e.g. number of I/O bits) and categorical features (e.g. node type - register, adder, multiplier, etc.). The node and edge encoders convert these categorical features to a learnable embedding.

Following the propagation through each multi-headed GATConv layer, a pooling layer compresses the learned embeddings from each graph node into a single graph-level embedding. We opted to use a concatenated pooling function that uses mean pooling and max pooling.

$$h_G^{\text{mean}} = \frac{1}{|V_G|}\sum_{i\in V_G} h_i \qquad h_G^{\max}[k] = \max_{i\in V_G} h_i[k]$$

where V_G is the set of graph nodes and $h_i[k]$ is dimension k of the learned embedding of node i. Mean pooling alone

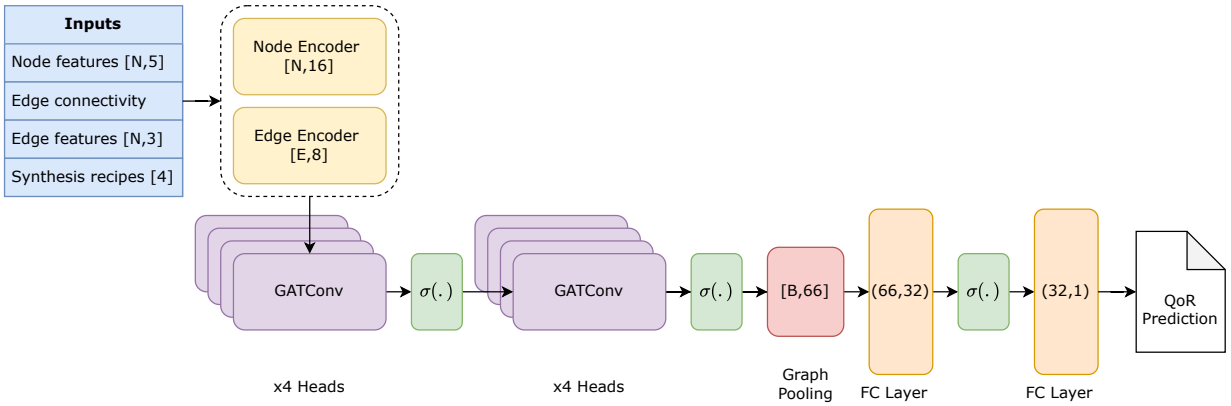


Figure 5. Block diagram of the QoRNet graph attention network.

tended to average out important features during WNS prediction, leading to poor performance on testing data, while max pooling alone overemphasized certain features from larger graphs in the training dataset, leading to early overfitting in the network. The concatenation adds two additional features: $\log(V_G)$ and $\log(E_G)$, the logarithm of the number of graph nodes and edges. Including dimensionality in the graph embedding helps to manage the difference in size between designs in the dataset.

Finally, the graph embedding propagates through two fully-connected layers before outputting a single prediction for either WNS or TNS. We use SmoothL1Loss as a loss function for GAT training

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\begin{cases}\frac{1}{2}(\hat{y}_i - y_i)^2, & \text{if } |\hat{y}_i - y_i| < 1\\ |\hat{y}_i - y_i| - \frac{1}{2}, & \text{otherwise}\end{cases}$$

SmoothL1Loss acts like MSE loss for prediction error close to 0 (< 1) and like L1 loss for prediction error ≥ 1. This limits the effect of outliers with large prediction errors skewing the training process. Empirically, we observed that SmoothL1Loss provided higher accuracy than MSELoss.

4.3. Dataset Collection

After Yosys generates the AST and synthesizes a design, the optimized netlist is exported to OpenROAD, which performs floorplanning (layout and power distribution), placement (placing all of the standard cells), clock tree synthesis (routing the clock network), and routing (wiring the control and data signals between standard cells). The post-routing timing reports from OpenROAD are used as the ground truth metrics for model training and evaluation.

Each implementation run can be quite lengthy and each design must be synthesized hundreds of times across different recipes, so it is necessary to parallelize the dataset generation flow as much as possible. For this, we use a slurm script to schedule synthesis job arrays of 50 runs at a time, using the Stokes CPU cluster on ARCC for fast evaluation.

In addition to feature representation and neural network architecture, we found that the process for training dataset collection posed a serious challenge. First, there is limited availability of large RTL benchmarks for the purposes of QoR prediction. Related works all rely on synthetic data generation to increase the amount of samples in their training dataset. For example, the training dataset from (Sengupta et al., 2022) includes 50 unique designs, but creates many additional samples by sweeping across various physical synthesis recipes. We adopt this methodology, performing recipe sweeps across max fanout, max switching capacitance, and max transition time. Many synthesis trials were required to find sweep values for each physical synthesis "recipe" that contributed meaningfully to the dataset (i.e. each unique recipe influenced the WNS or TNS).

Unlike other works, we included clock period as an additional recipe parameter. This constraint controls the degree to which a design is optimized for delay versus area. Initially, we synthesized all designs in our dataset under the same clock period constraint. This led to many larger designs having positive slack and TNS=0.000 (e.g. there were no timing violations). In the next iteration of our training dataset, we selected the clock period constraint on a per-design basis, sweeping across a range of clock period variations instead of absolute clock period. This provided more meaningful TNS data for model training. Without this augmentation, models tended to predict a value close to 0, which led to large absolute errors for any design with timing violations.

5. Experiment Setup

We pull RTL code from the ISCAS 1989 (Kajihara et al., 1996) and OpenCores (ope) benchmarks as training and

Source Benchmark	HDL Type	Number of Designs	Number of Recipes	Graph Nodes {Min, Median, Max}	Graph Edges {Min, Median, Max}
OpenCores	Verilog	6	288	{73, 205, 519}	{123, 363, 905}
ISCAS '89	Verilog	29	288	{23, 539, 23963}	{29, 882, 37859}

Table 2. Summary of circuit benchmark designs used for QoRNet training and evaluation.

Parameter	**Linear Regression**	**MLP**	**GAT**
Activation	-	ReLU	ReLU
Hidden Layers	-	64,32	32
Batch Size	1	1	32
# of Epochs	300	400	200
Learning Rate	10^{-2}	10^{-3}	10^{-4}
Dropout	-	0.1	0.1
Weight Decay	-	10^{-4}	10^{-4}
Loss Function	MSE	MSE	SmoothL1
Optimizer	Adam	Adam	Adam

Table 3. Parameters and hyperparameters used for model training.

testing data, synthesizing each across a spread of recipe parameters. For ground truth metrics, we synthesize and implement designs using the Nangate45 open-source PDK. Table 2 includes a summary of the benchmark sources used in QoRNet training. These benchmarks amount to ˜10,000 datapoints. Prediction accuracy is compared using the coefficient of determination

$$R^2 = \left[1 - \frac{\sum(\hat{y}_i - y_i)^2}{\sum(y_i - \bar{y})^2}\right]$$

where y_i represents the target value, $\hat{y}_i$ represents the model prediction, and $\bar{y}$ represents the mean target value in the dataset. The R^2 score is a measure of how well a model accounts for the variation in a dataset. The largest and best R^2 score is 1.0, which represents perfect accuracy, while a score of 0.0 is equivalent to a model that predicts the mean. In addition to the R^2 score, which captures the network's ability to model trends in a dataset, we use Mean Absolute Error (MAE) to capture the model's absolute accuracy:

$$MAE = \frac{1}{N}\sum|\hat{y}_i - y_i|$$

To ensure that predictions are a true reflection of QoRNet's ability to generalize to unseen data instead of a lucky testing set selection, we perform cross-validation training, then report the average R^2 and MAE across each folds. We use five cross-validation folds, which amounts to roughly an 80% training split.

Table 3 overviews the parameters used for model training, selected from our own repeated ablation studies. We implement early stopping criteria to halt training if the MAE or R^2 score has not improved for a predetermined amount of epochs. Additionally, we use Z-score normalization to normalize target data and feature vectors prior to model training.

6. Results

In this section, results are presented for WNS and TNS prediction. The linear regression and MLP models are used as baselines for evaluation against the GAT.

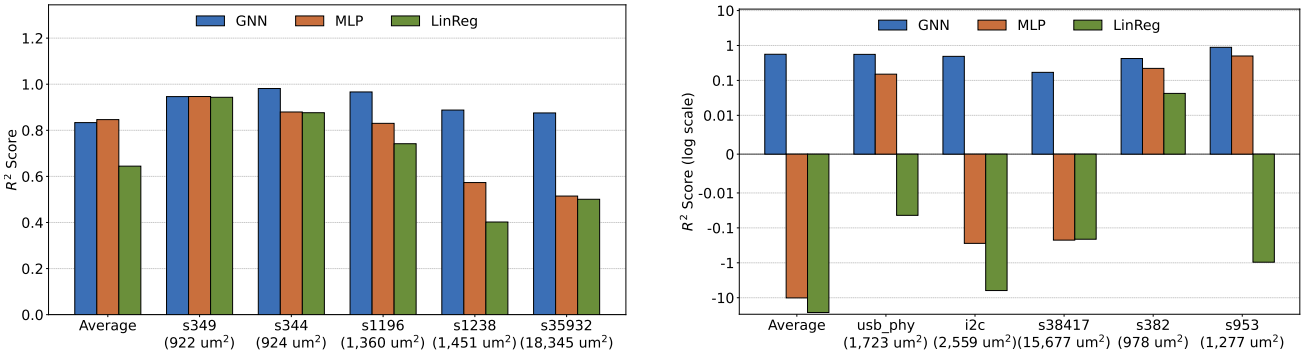
6.1. WNS Prediction

Figure 6 displays the resulting R^2 scores for training on a dataset of 35 designs from the OpenCores and ISCAS '89 benchmarks, with 288 recipes per design. Across all cross-validation folds, the MLP and GAT models exhibit comparable R^2 values of 83%, meaning that these two models account for around 83% of the variability seen in the testing data. The linear regression model's R^2 score is nearly 20 percentage points lower, suggesting that the impact of circuit structure on WNS is not well captured by a linear model. For smaller designs, seen in columns 1-3 of Figure 6, all three models explain variability across recipe values quite well; however, the GAT is much better at modeling the variability in larger designs such as s35932.

For certain test designs, the GAT model struggles to generalize WNS prediction from designs seen during training. Cross-validation training allows us to identify exactly which designs are causing issues. Figure 7a shows a scatter plot between predicted versus actual WNS, with points colored with respect to the per-design R^2 coefficient of determination. From the plot, we can identify issues with generalizing to s1423 and s38417 from the ISCAS '89 benchmark. s38417 has 23,013 nodes and 34,466 edges, whereas the global average is only 3123 nodes and 4868 edges. Meanwhile, s1423 has far fewer nodes and edges than the global average, suggesting there may be more complex interactions between graph nodes in this design. Overall, the R^2 coefficient of determination shows that the GAT model generalizes well to WNS prediction for the majority of designs in the dataset, with the global average R^2 score being brought down by a few outliers.

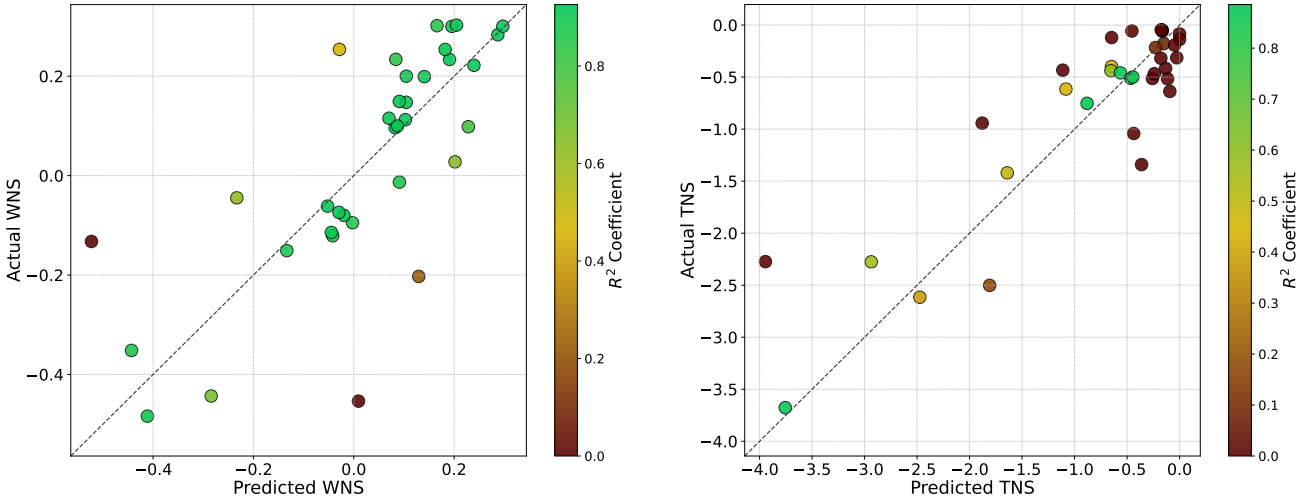
6.2. TNS Prediction

Unlike WNS prediction, which requires context from only the longest timing path through a design, TNS prediction



(a) Worst Negative Slack *(b)* Total Negative Slack

Figure 6. R^2 coefficient of determination results across linear regression, MLP, and GAT models.



(a) Worst Negative Slack *(b)* Total Negative Slack

Figure 7. Comparison of GAT prediction to actual target across all CV folds. The average WNS/TNS across all recipes is plotted, and R^2 is normalized against the global average R^2 score. WNS can be > 0 if a design meets timing (it then gives the smallest positive slack).

requires context from all paths in a design that are violating timing, making it a more complicated task. EDA tools will report a positive value for WNS, showing the smallest positive slack in a design, but they do not report positive TNS, meaning that TNS=0.000 if a design meets timing. In prior iterations of our training dataset, many of the TNS values were 0, resulting in low or even negative R^2 scores from all models.

For the final iteration of our training dataset, we include clock period on a per-design basis, leading to a spread of nonzero TNS values close to 0. As seen in Figure 6b, the vector-based models fail to learn any sort of representation for TNS prediction. Unlike WNS, which was a single scalar value that was strongly correlated with design size (and other scalar features included in each feature tensor), TNS requires context from *all* timing paths through a design. Because of this difference, feature vectors do not include enough context to accurately predict TNS, leading to average $R^2 < 0$ for all cross-validation folds. This means that the vector-based models would be better off predicting the mean target value from the entire dataset.

On the other hand, the average R^2 score for the GAT model is 55%, meaning it accounts for roughly half of the variability in the testing dataset designs and recipes. Between the 5 cross-validation folds, the minimum, median, and maximum R^2 scores are 31.5%, 56.3%, and 88.6%, respectively. We hypothesized that learning on the graph-based representation of a circuit would allow a model to more accurately model timing characteristics, and the difference in accuracy between the vector- and graph-based model supports this claim.

Another factor to consider is that the standard deviation of each target in our dataset is relatively low. As seen in Figure 7b, the prediction is less accurate for designs with larger TNS magnitude (more cumulative negative slack),

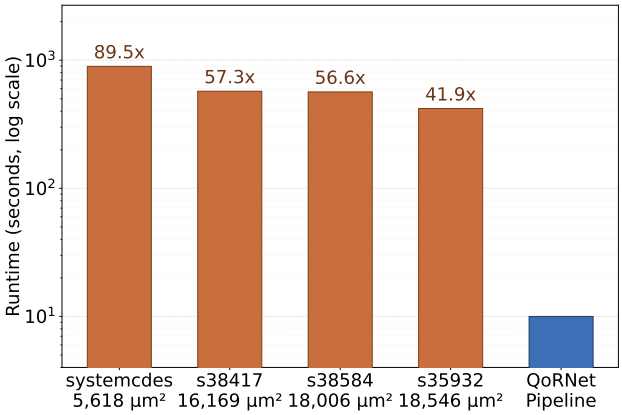


Figure 8. Speedup of QoRNet inference versus a full synthesis and implementation flow.

but the R^2 score is higher. For designs with TNS closer to 0, which is closer to the mean of the dataset, the R^2 score is lower, but the magnitude of the prediction error is also lower. For future experiments, the most high-impact change we can likely make is collecting a training dataset with a more normal distribution of TNS and WNS targets across designs.

6.3. Speedup

Finally, the end-to-end flow of QoRNet (AST generation with Yosys, graph processing, and model inference) can be completed in less than 10 seconds. This is a significant speedup over typical runtimes for commercial synthesis tools, whose runtime can be on the order of hours or days depending on the size of the design optimization effort. Even using an open-source flow (Yosys and OpenROAD), which offers significantly faster runtimes (albeit less accurate results) than commercial tools, the end-to-end QoRNet flow offers nearly two orders of magnitude speedup (See Figure 8).

7. Conclusion

In this work, we presented QoRNet, a set of deep learning models for post-implementation timing prediction. This project demonstrates that graph-based learning is useful in providing early timing estimates from an RTL-level representation. QoRNet can enhance the workflow of RTL designers as they iterate through different design choices at speed. Future work could further specialize the training of QoRNet models for specific classes of hardware, such as datapath components or control logic, or expand the prediction capabilities to area or dynamic power consumption. We could also collect target data with more of a normal distribution to prevent training on datasets with clustered designs and a few outliers (as seen in WNS prediction).

References

Opencores. URL https://opencores.org/.

Brody, S., Alon, U., and Yahav, E. How attentive are graph attention networks?, 2022. URL https://arxiv.org/abs/2105.14491.

Chowdhury, A. B., Tan, B., Karri, R., and Garg, S. Openabc-d: A large-scale dataset for machine learning guided integrated circuit synthesis. *CoRR*, abs/2110.11292, 2021. URL https://arxiv.org/abs/2110.11292.

Fey, M. and Lenssen, J. E. Fast graph representation learning with pytorch geometric, 2019. URL https://arxiv.org/abs/1903.02428.

Gandham, S., Walston, J., Samanta, S., Yin, L., Zheng, H., Lin, M., and Diamantidis, S. Circuitseer: Rtl post-pnr delay prediction via coupling functional and structural representation. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, ICCAD '24, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400710773. doi: 10.1145/3676536.3676668. URL https://doi.org/10.1145/3676536.3676668.

Kajihara, S., Kinoshita, K., Pomeranz, I., and Reddy, S. Combinationally irredundant iscas-89 benchmark circuits. In *1996 IEEE International Symposium on Circuits and Systems (ISCAS)*, volume 4, pp. 632–634 vol.4, 1996. doi: 10.1109/ISCAS.1996.542103.

Sengupta, P., Tyagi, A., Chen, Y., and Hu, J. How good is your verilog rtl code? a quick answer from machine learning. In *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pp. 1–9, 2022.

Wolf, C., Glaser, J., and Kepler, J. Yosys-a free verilog synthesis suite. In *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, volume 97, pp. 1–6, 2013.

Xu, C., Kjellqvist, C., and Wills, L. W. Sns's not a synthesizer: a deep-learning-based synthesis predictor. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ISCA '22, pp. 847–859, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450386104. doi: 10.1145/3470496.3527444. URL https://doi.org/10.1145/3470496.3527444.

8. Team Contribution Statement

Here, we describe each team member's contributions to the project.

8.1. Cory Brynds

Core responsibilities in the development of this project included automating the EDA pipeline (Yosys/OpenROAD scripts), selecting the training dataset, parallelizing the collection of ground truth data, building the Graph Attention Network architecture and training infrastructure, and iteratively improving the model parameters and dataset quality to improve model accuracy. Additionally, he wrote the project proposal document, prepared the in-class demo, and completed the following sections of the final report: introduction, background, implementation (intro, 4.2.2 and 4.3), experiment setup, results, and conclusion.

8.2. DeAndre Hendrix

DeAndre's contributions include developing and evaluating the baseline vector-based models. This includes constructing the preprocessing script for merging RTL feature data, synthesis recipes, and ground-truth metrics, running ablation studies for optimizing model hyperparameters, and writing Python scripts for synthesizing model performance data and generating figures. Notably, DeAndre also constructed the recipe improvement models, which were not elaborated upon in this report due to size constraints. These models introduce a novel contribution to existing work in the field by predicting optimal synthesis recipe improvements based on existing locally synthesized designs and can be found in the GitHub repository.

8.3. Youssef Samwel

Contributed to the paper by extracting global features from AST input and constructing graph representations from flattened AST data. Developed the graph traversal algorithm, PyTorch tensor conversion workflow, and visualization scripts.

8.4. Pablo Rodriguez

Contributions include background research of previous works, adapting the AI/ML pipeline to run on HPC systems, and preparing the Hugging Face Spaces live project demo, as well as minor assistance in model quality and accuracy evaluation. Rodriguez also wrote section 2, Related Work.

9. Use of External Code and AI Tools, Including LLMs

This section discloses the use of external code, tools, and AI usage throughout the development of QoRNet.

9.1. Use of External Code

The core components of this project (e.g. AST parsing, baseline models, and GNN) draw inspiration from (Sengupta et al., 2022). However, all components of the project were developed without external code, apart from some of the tool scripts used for Yosys and OpenROAD, which borrowed from: https://github.com/laiyao1/ArithTreeRL. Finally, RTL benchmarks were drawn from https://github.com/ispras/hdl-benchmarks.

9.2. Use of Artificial Intelligence

Artificial intelligence was used in the development of this project, including the use of LLMs (e.g. ChatGPT, Gemini) and coding agents (e.g. GPT Codex). All written materials, including proposals, presentations, and this report are the intellectual work of this group; however, LLMs were used for conceptual understanding of the related works associated with this paper.

Coding agents were primarily used for tasks unrelated to the intellectual merit of this project. This includes:

- Creating plots for interpreting results (with directed instructions for how to visualize data)
- Handling I/O operations such as parsing configuration files, terminal logging, and writing diagnostic information to CSVs
- Inserting error handling and debugging runtime errors

Additionally, coding agents were used to suggest potential optimizations and improvements to our data processing/training process. However, any suggested implementations were always verified with manual implementation and empirical evaluation.